

Звіт проєкту з Дискретної математики

Тема: Коди Боуза–Чоудхурі–Хоквінгема (БСН)

Киба Максим

Ришко Данило

Артем Сергійчук

Іван Богатирьов

8 травня 2026 р.

Український Католицький Університет

Факультет прикладних наук

II семестр, комп'ютерний проєкт

Зміст

1 Вступ	3
1.1 Мета проєкту	3
1.2 Постановка задачі	3
1.3 Сфери застосування БСН-кодів	3
2 Математичні основи БСН-кодування	4
2.1 Скінченні поля	4
2.1.1 Побудова $GF(2^m)$ через незвідний поліном	4
2.1.2 Таблиця елементів $GF(2^3)$	4
2.1.3 Операції в $GF(2^m)$	5
2.2 Мінімальні поліноми та циклотомічні класи	5
2.2.1 Циклотомічні класи за Фробеніусом	5
2.2.2 Мінімальний поліном	5
2.3 Побудова БСН-коду	5
2.4 Кодування	6
2.5 Перевірочна матриця	6
2.6 Декодування. Синдроми	6
2.7 Поліном локаторів помилок та алгоритм Берлекемпа–Мессі	7
2.7.1 Поліном локаторів помилок	7
2.7.2 Рівняння ключа	7
2.7.3 Алгоритм Берлекемпа–Мессі	7
2.8 Пошук коренів. Алгоритм Чена	7
2.9 Симуляція каналу та ймовірність помилки декодування	8
2.9.1 Модель каналу BSC	8
2.9.2 Ймовірність помилки декодування	8

3	Програмна реалізація	8
3.1	Технологічний стек	8
3.2	Структура каталогів	8
3.3	Модулі core/	9
4	Інтерфейси користувача	10
4.1	Командний інтерфейс	10
4.2	Веб-дашборд (Streamlit)	10
5	Експериментальні результати	10
5.1	Методика	10
5.2	Результати	11
5.3	Аналіз	11
5.4	Демонстрація передачі тексту	11
6	Розподіл ролей у команді	12
7	Висновки	12
8	Джерела	13

1 Вступ

1.1 Мета проєкту

Метою проєкту є побудова повноцінного симулятора бінарних BCH-кодів з нуля: математичної моделі (поле Галуа $GF(2^m)$, поліноміальна арифметика, алгоритм Берлекемпа–Мессі), кодувальника та декодера, моделі шумного каналу та інтерактивних інтерфейсів для демонстрації коректуючої здатності коду.

Ключові навчальні цілі:

- опрацювати теорію скінченних полів та циклічних кодів на практичному прикладі;
- самостійно реалізувати алгоритми обчислень у $GF(2^m)$ без використання спеціалізованих бібліотек;
- кількісно оцінити коректуючу здатність коду залежно від параметрів (m, t) та рівня шуму в каналі;
- продемонструвати ефект завадостійкого кодування на прикладі передачі довільного тексту.

1.2 Постановка задачі

Передача даних бінарним симетричним каналом з імовірністю спотворення біта p призводить до появи помилок. Без захисту повідомлення довжини k біт буде спотворено з імовірністю $1 - (1 - p)^k$, що швидко прямує до одиниці зі зростанням k або p . BCH-код (n, k, t) додає $n - k$ перевіірочних бітів і гарантовано виправляє до t помилок у блоці довжини $n = 2^m - 1$, що дозволяє знизити ефективну ймовірність помилки повідомлення на кілька порядків.

Задача проєкту: реалізувати повний стек — від арифметики поля до користувацького інтерфейсу — а потім емпірично виміряти, як параметри коду впливають на надійність відновлення.

1.3 Сфери застосування BCH-кодів

BCH-коди історично використовуються там, де канал має помірний рівень випадкового шуму та необхідне гарантоване виправлення невеликої кількості помилок:

- супутниковий зв'язок та глибокий космос (Voyager, GPS-навігація);
- оптичні носії (CD/DVD, штрих-коди QR як частковий випадок — коди Ріда–Соломона);
- NAND-flash-пам'ять у SSD-накопичувачах;
- завадостійке зберігання у файлових системах та архівах.

Розуміння BCH-кодів є відправною точкою для вивчення сучасніших схем — LDPC та полярних кодів.

2 Математичні основи ВСН-кодування

2.1 Скінченні поля

Означення 2.1. *Поле* Галуа $\text{GF}(q)$ називається скінченне поле з q елементами. Таке поле існує тоді й тільки тоді, коли $q = p^m$, де p – просте число, $m \in \mathbb{N}$.

У теорії ВСН-кодів використовуються поля виду $\text{GF}(2^m)$, де характеристика дорівнює 2.

2.1.1 Побудова $\text{GF}(2^m)$ через незвідний поліном

Означення 2.2. Поліном $p(x) \in \text{GF}(2)[x]$ степеня m називається *незвідним* над $\text{GF}(2)$, якщо він не розкладається в добуток поліномів меншого степеня над $\text{GF}(2)$.

Твердження 2.1. Нехай $p(x) \in \text{GF}(2)[x]$ – незвідний поліном степеня m . Тоді факторкільце

$$\text{GF}(2^m) \cong \text{GF}(2)[x] / (p(x))$$

є полем з 2^m елементами.

Кожен елемент $\text{GF}(2^m)$ представляється як клас еквівалентності полінома степеня менше m над $\text{GF}(2)$:

$$a_{m-1}\alpha^{m-1} + a_{m-2}\alpha^{m-2} + \dots + a_1\alpha + a_0, \quad a_i \in \text{GF}(2),$$

де α – примітивний елемент, тобто корінь $p(x)$: $p(\alpha) = 0$.

Зауваження 2.1. Умова $p(\alpha) = 0$ породжує рекурентне співвідношення

$$\alpha^m = - \sum_{i=0}^{m-1} c_i \alpha^i,$$

де c_i – коефіцієнти $p(x)$. У $\text{GF}(2)$ маємо $-1 = 1$, тому редукції виконуються через XOR. Це дозволяє виразити будь-яку степінь α^k , $k \geq m$, як поліном степеня менше m .

2.1.2 Таблиця елементів $\text{GF}(2^3)$

Приклад 2.1. Нехай $m = 3$, $p(x) = x^3 + x + 1$. Перевірка незвідності: $p(0) = 1 \neq 0$, $p(1) = 1 \neq 0$ – коренів у $\text{GF}(2)$ немає, отже $p(x)$ незвідний. Тоді $\alpha^3 = \alpha + 1$, і елементи поля мають вигляд:

Степінь	Поліном	Вектор (a_2, a_1, a_0)
0	0	000
α^0	1	001
α^1	α	010
α^2	α^2	100
α^3	$\alpha + 1$	011
α^4	$\alpha^2 + \alpha$	110
α^5	$\alpha^2 + \alpha + 1$	111
α^6	$\alpha^2 + 1$	101

Зауважимо, що $\alpha^7 = 1 = \alpha^0$, тобто $\text{ord}(\alpha) = 7 = 2^3 - 1$.

2.1.3 Операції в $\text{GF}(2^m)$

- **Додавання:** покоефіцієнтний XOR.
- **Множення:** множення поліномів з редукцією за $\text{mod } p(x)$.
- **Обернений елемент:** $(\alpha^i)^{-1} = \alpha^{2^m-1-i}$, що випливає з малої теореми Ферма для полів: $\alpha^{2^m-1} = 1$.

2.2 Мінімальні поліноми та циклотомічні класи

2.2.1 Циклотомічні класи за Фробеніусом

Означення 2.3. 2-циклотомічний клас C_i за модулем $n = 2^m - 1$ визначається як

$$C_i = \{i, 2i, 4i, \dots, 2^{s-1}i\} \pmod{n},$$

де s – мінімальне натуральне число таке, що $2^s i \equiv i \pmod{n}$.

Зауваження 2.2. Обґрунтування: відображення Фробеніуса $\varphi : x \mapsto x^2$ є автоморфізмом поля $\text{GF}(2^m)$ над $\text{GF}(2)$. Якщо α^i є коренем деякого полінома $f(x) \in \text{GF}(2)[x]$, то $(\alpha^i)^2 = \alpha^{2i}$ також є коренем $f(x)$. Отже, елементи одного циклотомічного класу є взаємно спряженими над $\text{GF}(2)$.

Приклад 2.2. Для $m = 3$, $n = 7$:

$$C_0 = \{0\}, \quad C_1 = \{1, 2, 4\}, \quad C_3 = \{3, 6, 5\}.$$

2.2.2 Мінімальний поліном

Означення 2.4. Мінімальним поліномом елемента $\alpha^i \in \text{GF}(2^m)$ над $\text{GF}(2)$ називається поліном найменшого степеня з коефіцієнтами в $\text{GF}(2)$, коренем якого є α^i .

Твердження 2.2. Мінімальний поліном $m_i(x)$ елемента α^i визначається формулою

$$m_i(x) = \prod_{j \in C_i} (x - \alpha^j) \in \text{GF}(2)[x].$$

Зокрема, $\deg m_i = |C_i|$.

Приклад 2.3.

$$m_1(x) = (x - \alpha)(x - \alpha^2)(x - \alpha^4) = x^3 + x + 1.$$

2.3 Побудова ВСН-коду

Означення 2.5. Примітивний ВСН-код з параметрами (n, k, t) над $\text{GF}(2)$ – це циклічний код довжиною $n = 2^m - 1$, що виправляє до t помилок, із породжуючим поліномом

$$g(x) = (m_1(x), m_2(x), \dots, m_{2t}(x)).$$

Оскільки елементи одного циклотомічного класу мають спільний мінімальний поліном, маємо явну факторизацію:

$$g(x) = \prod_{i \in C_1 \cup C_2 \cup \dots \cup C_{2t}} (x - \alpha^i).$$

Твердження 2.3 (Проектуюча властивість). Для кожного $j \in \{1, 2, \dots, 2t\}$:

$$g(\alpha^j) = 0.$$

Кількість інформаційних бітів: $k = n - \deg g(x)$.

Приклад 2.4. Для $\text{GF}(2^3)$, $t = 1$: $g(x) = x^3 + x + 1$, звідки $\deg g = 3$, $k = 7 - 3 = 4$.
Для $t = 3$: $g(x) = m_1(x) \cdot m_3(x) = x^6 + x^4 + x^2 + x + 1$, $k = 1$.

2.4 Кодування

Означення 2.6. Систематичне кодування. Нехай $u(x) \in \text{GF}(2)[x]$, $\deg u < k$ – інформаційний поліном. Кодове слово визначається як

$$c(x) = u(x) \cdot x^{n-k} + r(x),$$

де $r(x)$ – залишок від ділення $u(x) \cdot x^{n-k}$ на $g(x)$:

$$u(x) \cdot x^{n-k} = q(x) \cdot g(x) + r(x), \quad \deg r < n - k.$$

У $\text{GF}(2)$ маємо $-r(x) = r(x)$, тому $c(x) = u(x) \cdot x^{n-k} + r(x)$.

Твердження 2.4. $c(x)$ є кодовим словом, тобто $g(x) \mid c(x)$, і, отже, $c(\alpha^j) = 0$ для всіх $j = 1, \dots, 2t$.

2.5 Перевірочна матриця

Означення 2.7. Перевірочна матриця BCH-коду – матриця H над $\text{GF}(2^m)$ розміру $2t \times n$:

$$H = \begin{pmatrix} 1 & \alpha & \alpha^2 & \dots & \alpha^{n-1} \\ 1 & \alpha^2 & \alpha^4 & \dots & \alpha^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{2t} & \alpha^{4t} & \dots & \alpha^{2t(n-1)} \end{pmatrix}.$$

Це матриця Вандермонда за вузлами $1, \alpha, \alpha^2, \dots, \alpha^{n-1}$.

Твердження 2.5. Вектор $c = (c_0, c_1, \dots, c_{n-1}) \in \text{GF}(2)^n$ є кодовим словом тоді й тільки тоді, коли

$$c \cdot H^T = \mathbf{0}.$$

При розгортанні кожного елемента поля в m -бітовий вектор розмір H стає $(m \cdot 2t) \times n$.

2.6 Декодування. Синдроми

Нехай $r(x) = c(x) + e(x)$, де $e(x)$ – вектор помилок.

Означення 2.8. Синдром S_j визначається як

$$S_j = r(\alpha^j), \quad j = 1, 2, \dots, 2t.$$

Оскільки $c(\alpha^j) = 0$, маємо $S_j = e(\alpha^j)$. Нехай помилки розміщені на позиціях $i_1, i_2, \dots, i_\nu$. Тоді:

$$S_j = \sum_{\ell=1}^{\nu} \alpha^{j \cdot i_\ell} = \sum_{\ell=1}^{\nu} \beta_\ell^j,$$

де $\beta_\ell = \alpha^{i_\ell}$ називаються *локатором помилки* на позиції i_ℓ .

2.7 Поліном локаторів помилок та алгоритм Берлекемпа–Мессі

2.7.1 Поліном локаторів помилок

Означення 2.9. Поліном локаторів помилок визначається як

$$\Lambda(x) = \prod_{\ell=1}^{\nu} (1 - \beta_{\ell} x) = 1 + \Lambda_1 x + \Lambda_2 x^2 + \cdots + \Lambda_{\nu} x^{\nu}.$$

Корені $\Lambda(x)$ – це $\beta_{\ell}^{-1} = \alpha^{-i_{\ell}}$. Знаючи корені, знаходимо позиції помилок.

2.7.2 Рівняння ключа

З визначення синдромів і поліному $\Lambda(x)$ виводяться *рівняння ключа*:

$$S_j + \Lambda_1 S_{j-1} + \Lambda_2 S_{j-2} + \cdots + \Lambda_{\nu} S_{j-\nu} = 0, \quad j = \nu + 1, \nu + 2, \dots, 2t.$$

Це лінійна система відносно невідомих $\Lambda_1, \dots, \Lambda_{\nu}$.

2.7.3 Алгоритм Берлекемпа–Мессі

Алгоритм Берлекемпа–Мессі (ВМА) знаходить поліном $\Lambda(x)$ мінімального степеня, що задовольняє рівнянням ключа, за $O(t^2)$ операцій у полі. Стан алгоритму: $\Lambda(x)$, допоміжний поліном $B(x)$, крок останньої зміни p .

Ініціалізація: $\Lambda(x) \leftarrow 1, B(x) \leftarrow 1, p \leftarrow 0$.

Ітерація $r = 1, 2, \dots, 2t$:

1. Обчислити *нев’язку*:

$$\delta_r = S_r + \sum_{i=1}^{\deg \Lambda} \Lambda_i \cdot S_{r-i}.$$

2. Якщо $\delta_r = 0$: перейти до кроку 3.

Якщо $\delta_r \neq 0$: оновити

$$\Lambda^{\text{new}}(x) \leftarrow \Lambda(x) - \delta_r \cdot \delta_p^{-1} \cdot x^{r-p} \cdot B(x).$$

Якщо $2 \deg \Lambda \leq r - 1$: встановити $B(x) \leftarrow \Lambda(x)$, $\delta_p \leftarrow \delta_r$, $p \leftarrow r$, $\Lambda(x) \leftarrow \Lambda^{\text{new}}(x)$.

Інакше: $\Lambda(x) \leftarrow \Lambda^{\text{new}}(x)$.

3. Замінити $B(x) \leftarrow x \cdot B(x)$.

2.8 Пошук коренів. Алгоритм Чена

Маючи $\Lambda(x)$, необхідно знайти його корені в $\text{GF}(2^m)$.

Означення 2.10 (Пошук Чена). Послідовно перевіряється кожен ненульовий елемент поля:

$$\text{для } i = 0, 1, \dots, n - 1 : \quad \Lambda(\alpha^{-i}) \stackrel{?}{=} 0.$$

Якщо $\Lambda(\alpha^{-i}) = 0$, то i є позицією помилки.

Зауваження 2.3. Обчислення $\Lambda(\alpha^{-i})$ для послідовних i може бути організоване рекурентно, оскільки $\alpha^{-i-1} = \alpha^{-1} \cdot \alpha^{-i}$, що зменшує кількість множень.

Виправлення помилок. Для бінарних кодів ($e_i \in \text{GF}(2)$) виправлення зводиться до інвертування бітів:

$$\hat{c}_i = r_i \oplus 1 \quad \text{для кожного } i \in \{i_1, \dots, i_{\nu}\}.$$

2.9 Симуляція каналу та ймовірність помилки декодування

2.9.1 Модель каналу BSC

Означення 2.11. *Симетричний бінарний канал (BSC) з параметром $p \in [0, 1]$ – це модель, в якій кожен переданий біт незалежно спотворюється з ймовірністю p .*

2.9.2 Ймовірність помилки декодування

Твердження 2.6. Ймовірність того, що BCH(n, k, t)-декодер не зможе виправити помилки (тобто кількість помилок перевищує t):

$$P_e = \sum_{i=t+1}^n \binom{n}{i} p^i (1-p)^{n-i}.$$

При значеннях p , малих відносно критичного порогу, P_e спадає різко зі збільшенням t – це і є характерна “водоспадна” крива (waterfall curve) на графіку залежності BER після декодування від BER каналу.

3 Програмна реалізація

3.1 Технологічний стек

Проект реалізовано на Python 3.13 без використання спеціалізованих бібліотек кодування (galois, komm тощо). Вся арифметика поля, поліномів і алгоритми кодування/декодування написані з нуля. Зовнішні залежності:

- matplotlib – побудова графіків;
- streamlit – інтерактивний веб-дашборд.

3.2 Структура каталогів

```
bch-simulator/
|-- core/
| |-- galois.py # GF(2^m)
| |-- polynomial.py # GF(2) GF(2^m)
| |-- bch.py # BCH-, --
| |-- channel.py #
| '-- text_codec.py # BCH
|-- apps/
| |-- cli.py # CLI: encode, decode, simulate, plot, simulate-text
| '-- web/app.py # Streamlit-
|-- analysis/
| '-- benchmarks.py #
|-- examples/
| |-- quickstart.py #
| '-- plots/ #
|-- requirements.txt
|-- Makefile
'-- README.md
```


3.3 Модулі core/

galois.py. Реалізує клас `GaloisField(m)`, який конструює поле $\text{GF}(2^m)$ для $m \in [2, 10]$. Використовуються попередньо обчислені примітивні поліноми. Будуються таблиці *antilog* ($\alpha^i \mapsto$ векторне представлення) та *log* (зворотна) розміру $O(2^m)$, що дає $O(1)$ -операції множення та ділення. Метод `minimal_polynomial(a)` обчислює мінімальний поліном елемента шляхом генерації циклотомічного класу та розкриття добутку $\prod (x - a^{2^i})$.

polynomial.py. Клас `Polynomial` зберігає коефіцієнти у порядку зростання степеня. Підтримує додавання, множення, ділення з остачею як над $\text{GF}(2)$, так і над $\text{GF}(2^m)$ (через параметр `field`). Окрема функція `lcm_poly_gf2` обчислює НСК списку поліномів через алгоритм Евкліда – саме вона викликається при побудові породжуючого полінома $g(x)$.

bch.py. Основний клас `BCHCode(m, t)`:

- конструктор будує породжуючий поліном через об'єднання циклотомічних класів і обчислює параметри (n, k) ;
- метод `encode(message)` реалізує систематичне кодування через ділення з остачею;
- метод `decode(received)` виконує повний пайплайн: обчислення синдромів, ВМА, пошук Чена, виправлення; повертає структуру `DecodeResult` з полями `success`, `decoded_codeword`, `decoded_message`, `error_positions`, `syndromes`, `error_locator`;
- методи `generator_matrix()` і `parity_check_matrix()` – для теоретичного аналізу.

channel.py. Клас `BinarySymmetricChannel(p, seed)` моделює BSC: метод `transmit(bits)` повертає кортеж `(received, error_positions)`, де `received` – спотворений вектор, а `error_positions` – список індексів, у яких відбувся фліп. Параметр `seed` забезпечує відтворюваність експериментів.

text_codec.py. Розширення для передачі довільного тексту:

- `text_to_bits(text)` – UTF-8 кодування у бітовий потік (MSB-first);
- `encode_text(text, bch)` – розбиття бітового потоку на блоки по k біт, кодування кожного блоку у n -бітове кодове слово; повертає список кодових слів і початкову довжину для подальшого зрізання паддінгу;
- `decode_text(codewords, bch, original_length)` – декодування кожного блоку, конкатенація результатів, перетворення назад у текст з підрахунком успіхів і невдач.

4 Інтерфейси користувача

4.1 Командний інтерфейс

Файл `apps/cli.py` реалізує п'ять команд через `argparse`:

```
python -m apps.cli encode --m 5 --t 3 --message "1010..."
python -m apps.cli decode --m 5 --t 3 --received "1010..."
python -m apps.cli simulate --m 5 --t 3 --ber 0.05 --seed 42
python -m apps.cli plot --m 5 --t 3 --output curve.png
python -m apps.cli simulate-text --m 5 --t 3 --text "Hello" --ber 0.02
```

4.2 Веб-дашборд (Streamlit)

Файл `apps/web/app.py` надає інтерактивний інтерфейс із чотирма вкладками:

1. **Encoder/Decoder** – ручне введення повідомлення, інтерактивна ін'єкція помилок на конкретні позиції, спостереження за результатом декодера в реальному часі.
2. **Channel Simulator** – запуск пакету повідомлень через BSC і агреговані метрики.
3. **Performance** – побудова кривої успішності декодування vs BER з налаштовуваною дискретизацією.
4. **Text Transmission** – передача довільного тексту через канал; візуалізація трьох версій – оригінал, текст без декодування, текст після BCH-декодування.

5 Експериментальні результати

5.1 Методика

Експеримент будується так: для фіксованого коду $BCH(n, k, t)$ і набору значень BER $\{p_1, p_2, \dots\}$ для кожного p_i виконується T незалежних випробувань, у кожному з яких:

1. генерується випадкове повідомлення $\mathbf{u}$ довжини k ;
2. обчислюється кодове слово $\mathbf{c} = \text{encode}(\mathbf{u})$;
3. кожен біт $\mathbf{c}$ незалежно спотворюється з імовірністю p_i ;
4. декодер обробляє отриманий вектор; результат класифікується як *успіх*, якщо `decoded_message == u`.

Кількість успіхів, поділена на T , дає емпіричну оцінку ймовірності успішного декодування.

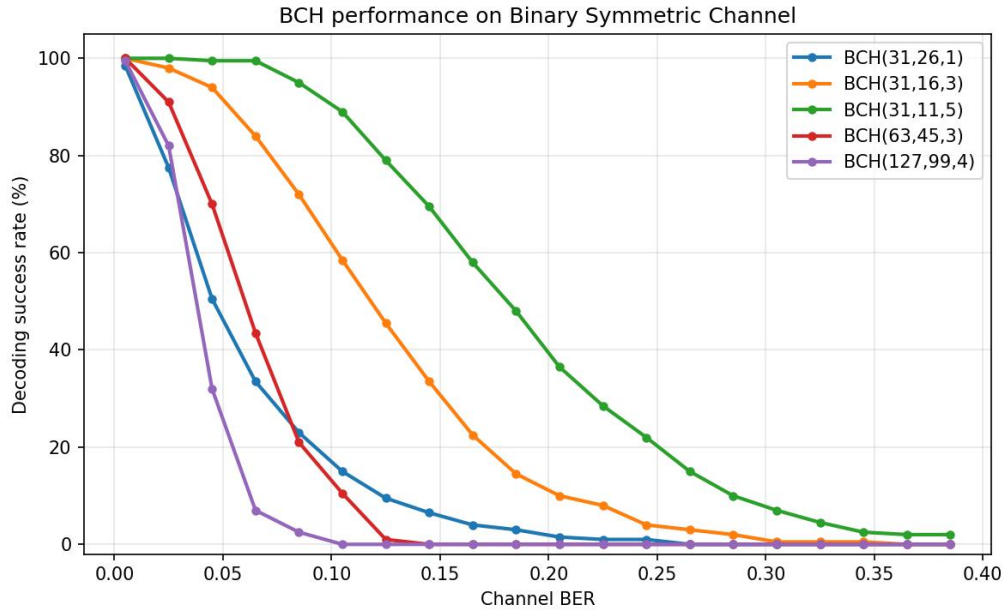


Рис. 1: Успішність декодування як функція BER каналу для п'яти BCH-кодів. $T = 200$ випробувань на точку, 20 значень BER у діапазоні $[0.005, 0.385]$.

5.2 Результати

5.3 Аналіз

Із рис. 1 спостерігаємо такі закономірності:

- **Ефект параметра t .** Криві мають характерну форму “водоспаду”: при низькому BER ($p < t/n$) успішність наближається до 100%, після чого падає до плато на рівні випадкового вгадування. Зі збільшенням t поріг водоспаду зсувається праворуч.
- **Tradeoff між t і кодовою швидкістю.** Збільшення t ціною зменшення k/n : для BCH(31, 26, 1) маємо швидкість 0.84, для BCH(31, 6, 7) – лише 0.19. У режимах з низьким BER невелике t ефективніше; у шумних каналах виправдане більше.
- **Залежність від n .** Для більших n (а отже, довших блоків) криві стають крутішими – довший блок дає кращу статистичну усередненість, але ціною більшої затримки кодування/декодування.
- **Складність декодування.** Емпірично декодування блоку BCH(127, 99, 4) виконується за десятки мілісекунд на стандартному CPU, що підтверджує практичність алгоритму Берлекемпа–Мессі ($O(t^2)$) у порівнянні з прямим розв'язком системи рівнянь ($O(t^3)$).

5.4 Демонстрація передачі тексту

Окрім графіків, було проведено демонстрацію в інтерактивній вкладці “Text Transmission”. При BER = 0.03 та кодів BCH(31, 16, 3) короткі повідомлення (10–20 символів)

майже завжди відновлюються повністю, тоді як без BCH-декодування у середньому 4–5 байтів спотворюються до нечитаних. Це наочно ілюструє практичну користь завадостійкого кодування.

6 Розподіл ролей у команді

Учасник	Внесок
Киба Максим	Поле Галуа $GF(2^m)$, поліноміальна арифметика, мінімальні поліноми
Ришко Данило	Кодувальник BCH, декодер з алгоритмом Берлекемпа–Мессі та пошуком Чена
Артем Сергійчук	Модель каналу BSC, бенчмарки, генерація графіків
Іван Богатирьов	CLI, Streamlit-дашборд, модуль кодування тексту, документація

Робочий процес: окремі гілки в Git для кожного модуля з подальшим merge у main через pull request після code review. Загальна структура зрозуміла з історії комітів.

7 Висновки

У ході проєкту:

1. Реалізовано з нуля повний стек BCH-кодування: арифметику $GF(2^m)$, поліноми, алгоритми кодування і декодування.
2. Підтверджено теоретичні передбачення: код $BCH(n, k, t)$ виправляє до t помилок, а ймовірність невдачі підпорядковується закону біноміального “водоспаду”.
3. Побудовано два інтерфейси (CLI та Streamlit), що дозволяють як автоматичні експерименти, так і інтерактивну демонстрацію.
4. Проведено серію експериментів, кількісно показано залежність успішності декодування від BER та параметрів коду.

Можливі напрямки розширення:

- декодер Сугіями (через розширений алгоритм Евкліда) як альтернатива ВМА;
- м’яке декодування з використанням алгоритму Чейза;
- недвійкові BCH-коди (зокрема коди Ріда–Соломона) над $GF(2^m)$ для $m > 1$ як алфавіт;
- канал AWGN з BPSK-модуляцією для оцінки в одиницях E_b/N_0 ;
- оптимізація на векторні операції (numpy/numba) для прискорення.

8 Джерела

1. Wikipedia. BCH code. https://en.wikipedia.org/wiki/BCH_code
2. Вихідний код проекту на GitHub: https://github.com/mr-amigo/diskret_comp_project_bch